Automata — DS preparation

Dr-Ing Ines Baccouche · 7 exercises covering every chapter · built from her Tutorial 1 (series.pdf), her slides and the class board · solutions at the end

How to use this in the time you have

Order	Exercise	Covers	Time
1–3	E1, E2, E3	RegEx notation, true/false, language → RegEx + DFA	done ✔
4 ★	E4 — aba / bab / bbb	explain a language, Glushkov, DFA, complement	20 min
5 ★	E5 — read 4 automata	automaton → language, deterministic / complete	15 min
6 ★★	E6 — (a|ba|bba)* (b|bb|ε)	the big one: Glushkov → NFA with ε column → DFA → minimisation → drawings, traces	35 min
7 ★★	E7 — slide 79 (ε-NFA)	NFA table with ε column → DFA table → drawing → minimal DFA	25 min

★ = do these first: they are the long DS-style questions. Work on paper, in her format (tables, Step0..., δ'([q0],a) = ...). When stuck, open the cheat sheet section given in each exercise, then continue. Check the solution only after you finish the exercise.

Exercises

E1 — Numbers (Ch1, RegEx notation)

Tutorial 1 · Ex 1 Cheat sheet: Ch1 §3 notation

Alphabet {0,...,9}, d = [0-9]. Give the RegEx of:

1. numbers of exactly three digits;
2. numbers that can be divided by 5;
3. numbers that start with 4 and end with 7;
4. numbers of three digits that do not start with 0.

E2 — True or false? (Ch1)

slides 27–29 style Cheat sheet: Ch1 §6 and §7

1. True or false? Justify with a rule (true) or a counter-example word (false):
   `(a*b)* = (a|b)*b` · `(ab)*a = a(ba)*` · `(a|b)* = (a*b*)*` · `(ab)* = a*b*`
2. R = `(a|b)*bb` and S = `b(a|b)*`. Give a word: in R not in S · in S not in R · in both · in neither.

E3 — RegEx and automaton of a language (Ch1 + Ch2)

Tutorial 1 · Ex 4 Cheat sheet: Ch1 §4, Ch3 §3

Over {a,b}, give the RegEx **and** a deterministic automaton of the words containing:

1. at least one a;
2. at most one a;
3. an even number of a.

E4 — Explain and build (Ch1 + Ch2 + Ch3)

★ DS **Tutorial 1 · Ex 2** Cheat sheet: Glushkov, Ch3 §4 (flip)

For each language, explain it in words and draw an automaton that recognises it using a construction method:

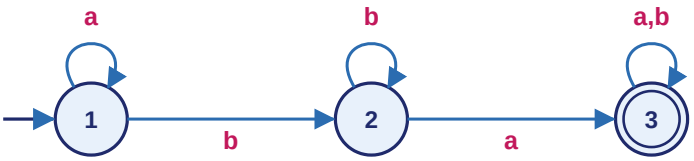
1. L denoted by `(aba)|(bab)`;
2. L denoted by `(aba)*|(bab)*`;
3. L = { u ∈ {a,b}* | u contains the factor bbb };
4. L = { u ∈ {a,b}* | u does not contain the factor bbb }. Also give its RegEx.

E5 — Give the language of each automaton (Ch2 + Ch3)

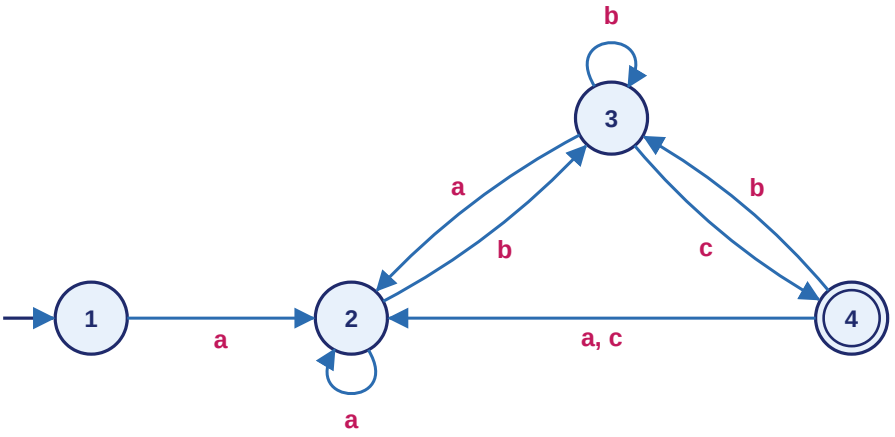
★ DS | Tutorial 1 · Ex 6 | Cheat sheet: Ch3 §1, Extra A (Arden)

For each automaton: describe the language in words, give a RegEx, and say if it is deterministic and complete.

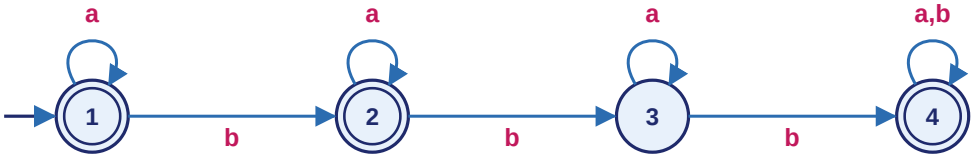
a)



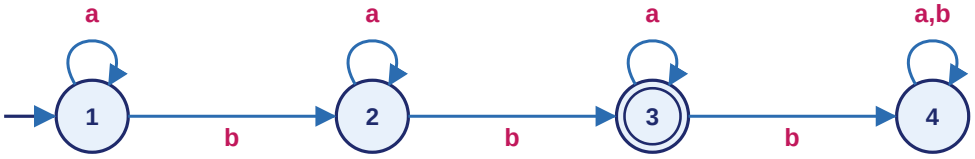
b) alphabet {a,b,c}



c)



d)



e) What is the link between c) and d)?

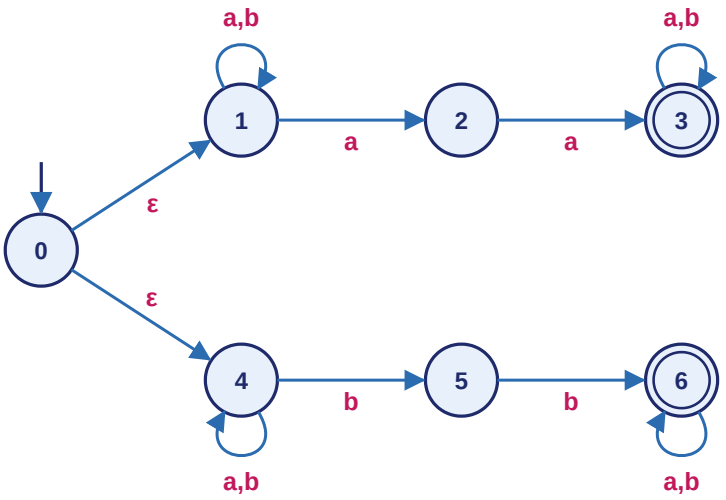
E6 — The big one: (a | ba | bba)* (b | bb | ε)

★★ DS · slide 61 + class board · Cheat sheet: Glushkov, NFA→DFA, Minimisation

1. **Glushkov:** number the letters (ε gets its own number, like on the board), give start letters, end letters and the successors table.
2. **NFA:** write the transition table with columns **a | b | ε**. Is it deterministic? Why?
3. **NFA → DFA:** convert it (start = 0 + its ε cell). Give the DFA table and draw it.
4. **Minimise** the DFA (Step0, Step1…) and draw the minimal automaton.
5. Trace **abbab** and **abbba** on the minimal DFA. What is the language, in one sentence?

E7 — Slide 79: create the corresponding DFA (NFA → DFA)

★★ DS · slide 79 (class photo) · Cheat sheet: NFA→DFA



1. Write the NFA table with columns **a | b | ε**. First row: `→0 | – | – | {1,4}`
2. Convert it to a DFA (start = 0 + its ε cell). Give the DFA table, mark the new states and the finals.
3. Draw the DFA.
4. Bonus: minimise it. What is the language?

Solutions

E1

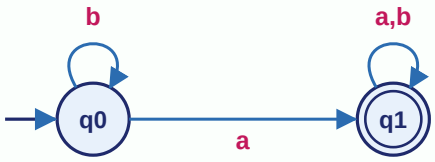
1. `\d{3}` = `ddd`
2. `\d*(0|5)` — her slide: (0-9)*(0|5). Any number ending with 0 or 5.
3. `4\d*7` (her slide: `4.*7`) — 47 ✔, 4 ✘, 417 ✔
4. `[1-9]\d{2}`

E2

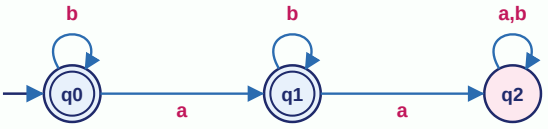
1. `(a*b)* = (a|b)*b` → **False**: ε ∈ (a*b)* but ε ∉ (a|b)*b (must end with b).
   `(ab)*a = a(ba)*` → **True**: sliding rule (rs)*r = r(sr)*. Both = {a, aba, ababa, ...}.
   `(a|b)* = (a*b*)*` → **True**: every word is a sequence of a's and b's, each letter is one a*b* block.
   `(ab)* = a*b*` → **False**: a ∈ a*b* but a ∉ (ab)* (also aab).
2. R = ends with bb, S = starts with b.
   In R not S: **abb** · in S not R: **b** · in both: **bb** · in neither: **a**

E3

1. At least one a: `b*a(a|b)*`

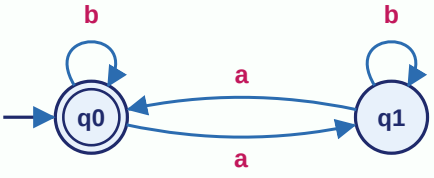


2. At most one a: `b* | b*ab*` = `b*(a|ε)b*`



q2 = dead state (a second a).

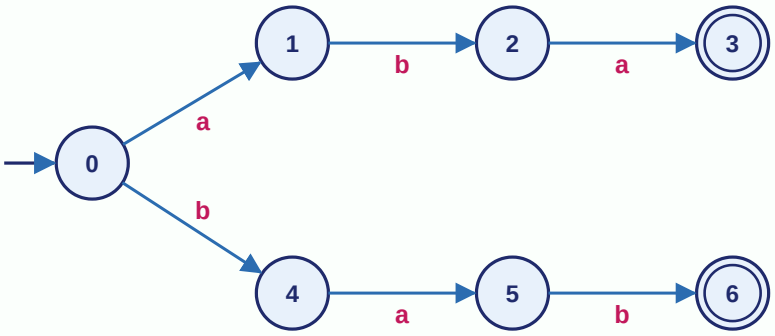
3. Even number of a: `b*(ab*ab*)*` = `(b|ab*a)*`



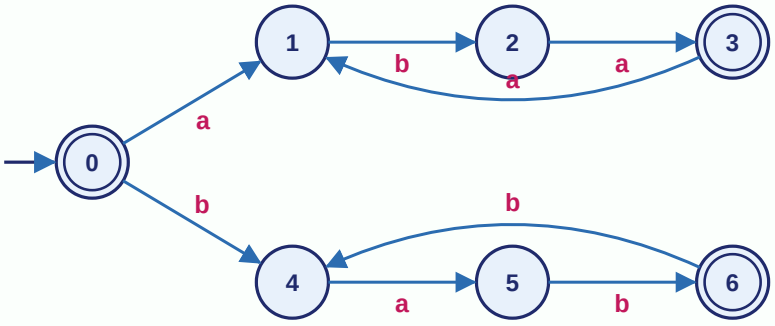
q0 = even, q1 = odd. ε is accepted (0 a's).

E4

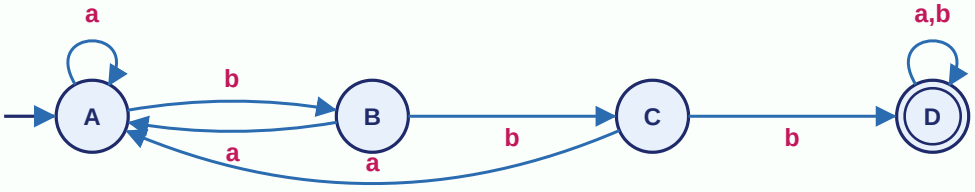
1. L = {aba, bab}: exactly two words. Glushkov: `a1 b2 a3 | b4 a5 b6`; First = {a1, b4}, Last = {a3, b6} ⇒ F = {3, 6}; successors a1→b2, b2→a3, b4→a5, a5→b6.



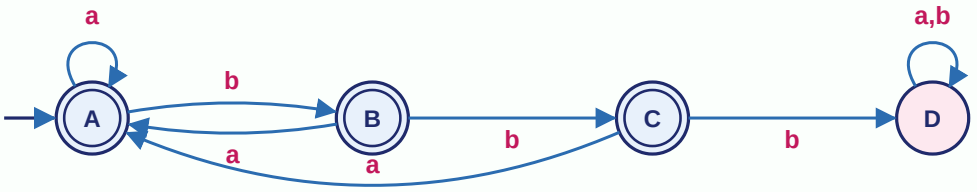
2. L = repetitions of aba only, or repetitions of bab only: {ε, aba, abaaba, ..., bab, babbab, ...}. No mixing: ababab (= aba·bab) ✘ — same idea as a*|b*.
Glushkov: `(a1 b2 a3)* | (b4 a5 b6)*`. Nullable ⇒ **0 ∈ F**. F = {0, 3, 6}. Successors: a1→b2, b2→a3, **a3→a1** (star), b4→a5, a5→b6, **b6→b4** (star).



3. Contains bbb: `(a|b)*bbb(a|b)*`. State = number of b's read in a row (A: 0, B: 1, C: 2, D: bbb found). An a sends back to A.



4. Complement of 3: the DFA is complete ⇒ **flip the states** (A, B, C final, D non-final = dead).



RegEx: `(a|ba|bba)*(ε|b|bb)` — exactly the class exercise of slide 61! Blocks a, ba, bba never make 3 b's; the end adds at most bb.

E5

a) Words over {a,b} that contain **ba**: `(a|b)*ba(a|b)*` = `a*bb*a(a|b)*`. Deterministic and complete.

b) What each state remembers: 2 = last letter a (or "bcc"), 3 = last letter b, 4 = ends with "bc". A c is only possible from 3 or 4, so **every c comes right after b or after "bc"**, never after a, and at most 2 c in a row.
Language: words over {a,b,c} that **start with a**, **end with bc**, where every block of c's follows a b and has length 1 or 2: `a(a|b|bc|bcc)*bc` = `a(a|b(ε|c|cc))*bc`. Words: abc, aabc, abbc, abcbc, abccbc · ✘ acbc (c after a), abccc.
Unambiguous, but **not complete** (1 has no b, c; 2 has no c) ⇒ by her definition it is **not a DFA** until you add a dead state for the missing arrows.

c) Number of b **different from 2** (0, 1, or at least 3): `a* | a*ba* | a*ba*ba*b(a|b)*`. Deterministic and complete.

d) Exactly two b: `a*ba*ba*`. State 4 is a dead state. Deterministic and complete.

e) Same automaton, final and non-final states flipped ⇒ d) is the **complement** of c).

> Method for any automaton: find what each state "remembers" (number of b's, last letter...), then follow start → final paths: loop = *, choice = |.

E6

1. Glushkov — `(a1 | b2 a3 | b4 b5 a6)* (b7 | b8 b9 | ε10)` · states 0 … 10

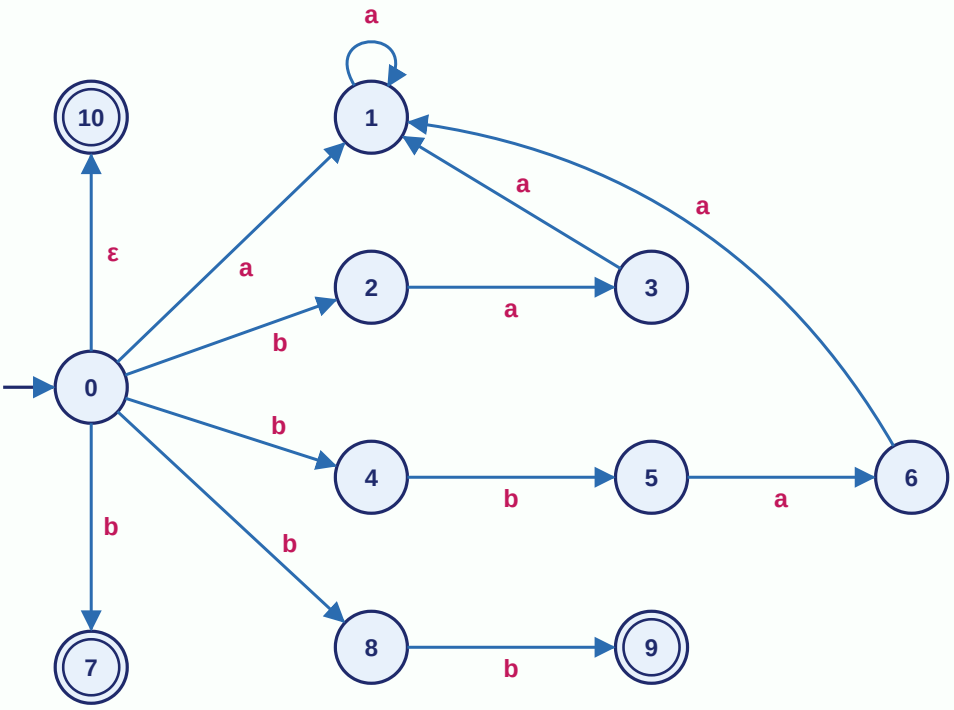
	successors
ϕ	a1 b2 b4 b7 b8 ε10
a1	a1 b2 b4 b7 b8 ε10
b2	a3
a3	a1 b2 b4 b7 b8 ε10
b4	b5
b5	a6
a6	a1 b2 b4 b7 b8 ε10
b7	ϕ
b8	b9
b9	ϕ
ε10	ϕ

- **Start letters** = a1, b2, b4 (start of a block) and b7, b8, ε10 (the star can be empty).
- **End letters** = b7, b9, ε10 ⇒ **F = {7, 9, 10}**.
- a1, a3, a6 end a block of the star ⇒ they loop back to a1, b2, b4 **or** leave the star to b7, b8, ε10.

2. NFA table (rows 0, 1, 3, 6 are identical):

δ	a	b	ε
→0	1	{2,4,7,8}	10
1	1	{2,4,7,8}	10
2	3	–	–
3	1	{2,4,7,8}	10
4	–	5	–
5	6	–	–
6	1	{2,4,7,8}	10
*7	–	–	–
8	–	9	–
*9	–	–	–
*10	–	–	–

Not deterministic: b goes to 4 states at once, there are ε-arrows, and many cells are empty.



1, 3, 6: same exits as 0 (see table)

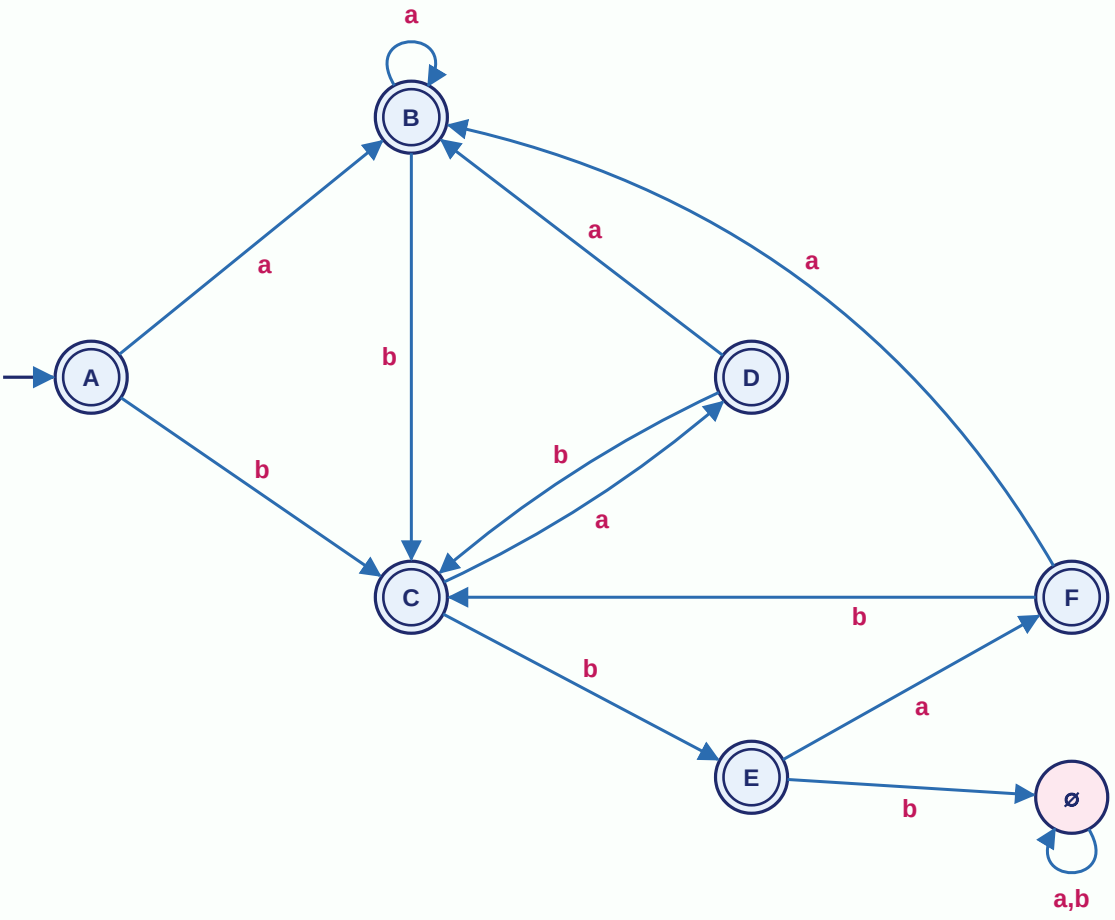
Drawing: all exits of 0 are drawn; 1, 3 and 6 have **the same exits as 0** (a→1, b→2,4,7,8, ε→10) — see the table.

3. NFA → DFA. Start = 0 + its ε cell = **[0,10]**. After each move, add the ε cell of the states you land on (1, 3, 6 → add 10).

```
δ'([0,10],a)    = {1}       + ε of 1 → [1,10]      ← new
δ'([0,10],b)    = {2,4,7,8}              → [2,4,7,8]   ← new
δ'([1,10],a)    = {1}       + 10         → [1,10]
δ'([1,10],b)    = {2,4,7,8}              → [2,4,7,8]
δ'([2,4,7,8],a) = 2→3       + ε of 3 → [3,10]      ← new
δ'([2,4,7,8],b) = 4→5, 8→9               → [5,9]       ← new
δ'([3,10],a)    = {1} + 10  → [1,10]    δ'([3,10],b) = [2,4,7,8]
δ'([5,9],a)     = 5→6       + ε of 6 → [6,10]      ← new
δ'([5,9],b)     = nothing                → ø (dead)
δ'([6,10],a)    = [1,10]                 δ'([6,10],b) = [2,4,7,8]
```

δ'	a	b
→*A = [0,10]	B	C
*B = [1,10]	B	C
*C = [2,4,7,8]	D	E
*D = [3,10]	B	C
*E = [5,9]	F	ø
*F = [6,10]	B	C
ø	ø	ø

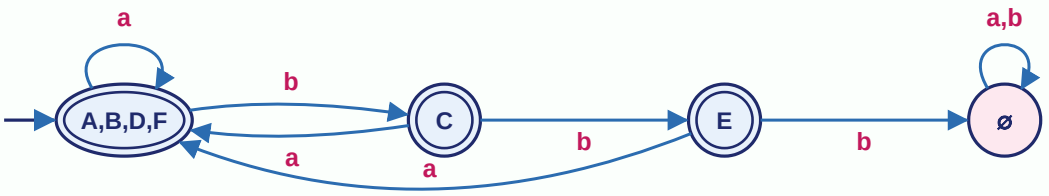
Final = contains 7, 9 or 10 ⇒ every state except ø.



4. Minimisation

```
Step0: (A,B,C,D,E,F) (⌀)                        finals / non-finals
Step1: E –b→ ⌀, the others –b→ a final  ⇒ (A,B,C,D,F) (E) (⌀)
Step2: C –b→ E, A,B,D,F –b→ C            ⇒ (A,B,D,F) (C) (E) (⌀)
Step3: A,B,D,F: a → (A,B,D,F), b → (C)   ⇒ same ⇒ stable
fusion: {A,B,D,F} = "A,B,D,F"   C   E   ⌀
```

δ	a	b
→*(A,B,D,F)	(A,B,D,F)	C
*C	(A,B,D,F)	E
*E	(A,B,D,F)	⌀
⌀	⌀	⌀



Meaning of each state: "A,B,D,F" = 0 b's in a row · C = 1 b · E = 2 b's in a row · ⌀ = bbb was read.

5. abbab: (A,B,D,F) –a→ (A,B,D,F) –b→ C –b→ E –a→ (A,B,D,F) –b→ C ✔ · abbba: … –b→ C –b→ E –b→ ⌀ ✘

Language: all words over {a,b} that **never contain bbb** — the same answer as E4.4!

> On the board she minimised the Glushkov table directly (groups {7,9,10} {0,1,3,6} {8} {2} {4} {5}). If the question says "minimise" right after Glushkov, write her version (in the cheat sheet). If it says "DFA, then minimise", write this one.

E7

1. NFA table

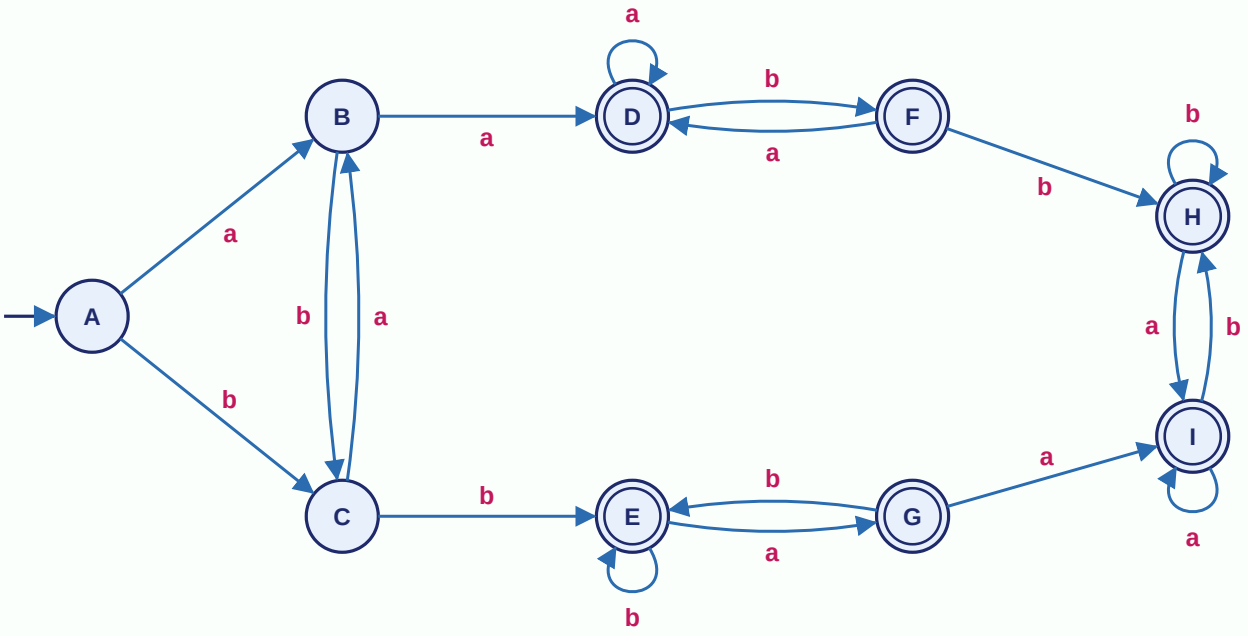
δ	a	b	ε
→0	–	–	{1,4}
1	{1,2}	1	–
2	3	–	–
*3	3	3	–
4	4	{4,5}	–
5	–	6	–
*6	6	6	–

2. DFA table — start = 0 + its ε cell = **[0,1,4]**

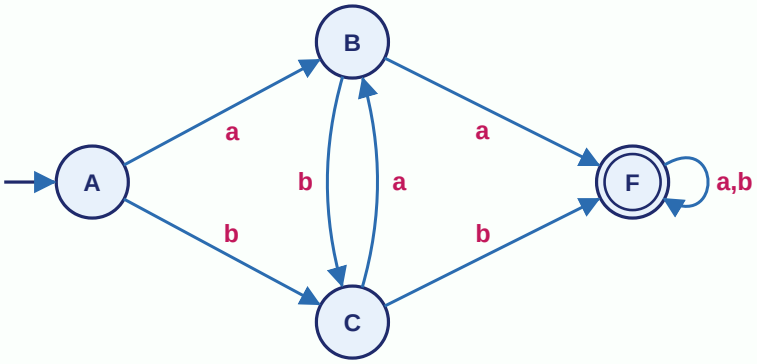
δ'	a	b
→A = [0,1,4]	[1,2,4] = B ← new	[1,4,5] = C ← new
B = [1,2,4]	[1,2,3,4] = D ← new	C
C = [1,4,5]	B	[1,4,5,6] = E ← new
*D = [1,2,3,4]	D	[1,3,4,5] = F ← new
*E = [1,4,5,6]	[1,2,4,6] = G ← new	E
*F = [1,3,4,5]	D	[1,3,4,5,6] = H ← new
*G = [1,2,4,6]	[1,2,3,4,6] = I ← new	E
*H = [1,3,4,5,6]	I	H
*I = [1,2,3,4,6]	I	H

Example cell: B with a → 1→{1,2}, 2→3, 4→4 ⇒ [1,2,3,4].
Final = contains 3 or 6.

3. Drawing



4. Minimal DFA: Step0: (D,E,F,G,H,I) (A,B,C) · Step1: B –a→ final, C –b→ final, A never ⇒ (D…I) (A) (B) (C) · D…I only go to D…I ⇒ fusion into one final state, drawn as "F" below (= D, E, F, G, H, I together).



A = start · B = last letter a · C = last letter b · F = found aa or bb. **Language: words that contain aa or bb** = `(a|b)*(aa|bb)(a|b)*`.